université
PARIS-SACLAY

RAPPORT DE TP

Architecture avancée

Auteur :

Gaëtan BOUCHY
Thomas ABOU JAOUDE
Victor BERCY

Enseignant :

M. HAMMAMI
M. GAC

Nous attestons que ce travail est original, qu'il est le fruit d'un travail commun en trinôme et qu'il a été rédigé de manière autonome.

À Orsay, le 16/12/2022

université
PARIS-SACLAY

Sommaire

Sommaire 2
TP3 - Simulateur de processeur MIPS 3
1. Etude du pipeline sur des programmes élémentaires 3
2. Comportement de programmes complexes 5
3. Ecriture de programmes assembleur 8
3.1 Premier programme 8
3.2 Deuxième programme 12

université PARIS-SACLAY

TP3 - Simulateur de processeur MIPS

1. Etude du pipeline sur des programmes élémentaires

Les résultats du premier tableau ont été réalisés sans l'usage des mécanismes d'envoi (forwarding) dans le processeur :

Fonctions / Carac-téristiques	sum.s	mul.s	sumf.s	mulf.s	divf.s	abs.s
Nombre de cycles	13	16	16	19	36	19
Nombre d'instructions	5	6	5	5	5	8
CPI	2.6	2.66	3.2	3.8	7.2	2.375
Nombre de cycles d'attente	4	6	7	10	27	7

Tableau 1 : Tableau des caractéristiques du pipeline en fonction de programmes élémentaires (sans forwarding)

Les résultats du second tableau ont été réalisés avec l'usage des mécanismes d'envoi (forwarding) dans le processeur :

Fonctions / Carac-téristiques	sum.s	mul.s	sumf.s	mulf.s	divf.s	abs.s
Nombre de cycles	10	11	13	16	33	16
Nombre d'instructions	5	6	5	5	5	8
CPI	2	1.833	2.6	3.2	6.6	2
Nombre de cycles d'attente	1	1	4	7	24	4

Tableau 2 : Tableau des caractéristiques du pipeline en fonction de programmes élémentaires (avec forwarding)

Nous remarquons qu'avec les mécanismes d'envoi (forwarding), le nombre de cycles, le nombre d'instructions, le CPI et le nombre de cycles d'attente diminuent fortement. En effet le forwarding permet de court-circuiter et d'aller chercher la donnée là où elle est présente.

Si nous regardons maintenant les latences entre les opérations entières et flottantes, nous remarquons des écarts assez importants :

Fonctions / Latences	sum.s	mul.s	sumf.s	mulf.s	divf.s	abs.s
Latence (sans forwarding)	4	6	7	10	27	7
Latence (avec forwarding)	1	1	4	7	24	4

Tableau 3 : Tableau des latences des opérations entières et flottantes avec et sans forwarding

Tout d'abord, les fonctions entières : sum et mul se réalisent en 4 et 6 cycles d'attente sans forwarding et en seulement 1 avec forwarding.

Puis les fonctions flottantes sumf et mulf se réalisent quant à elle en 7 et 10 cycles d'attente sans forwarding et en 4 et 7 avec forwarding. Cela est donc bien supérieur aux opérations entières.

Nous allons maintenant effectuer le programme double.s avec forwarding puis sans forwarding :

Fonctions / Carac-téristiques	double.s (sans forwarding)	double.s (avec forwarding)
Nombre de cycles	90	83
Nombre d'instructions	25	25
CPI	3.6	3.32
Nombre de cycles d'attente	61 (38 raw stalls + 23 divider not available)	54 (29 raw stalls + 23 divider not available + 2 memory not available)

Tableau 4 : Tableau des caractéristiques du pipeline pour le programme double.s

Entre la division et la multiplication, nous remarquons qu'il y a plusieurs unités de multiplication alors qu'il n'y en a qu'une seule pour la division. Sinon, ils utilisent le même schéma d'instruction : ld (load), ld (load), div ou mul, sd (store), halt.

Pour finir, l'ensemble de ces programmes assembleurs pourraient être optimisés par réordonnancement afin d'éviter au maximum les dépendances (de données : RAW, de nom : WAR, de sortie : WAW) et de réduire les cycles d'attente. Cela permettrait donc d'éviter les aléas de données c'est-à-dire le fait qu'une donnée ne soit pas disponible à cause de la latence du pipeline.

2. Comportement de programmes complexes

Nous allons maintenant étudier deux programmes : le premier est fibs.s qui réalise l'algorithme de Fibonacci pour les 12 premiers termes puis le second est daxpy.s qui comme son nom l'indique, réalise $y_i = a*x_i + y_i$.

Fonctions / Caractéristiques	fibs.s (sans forwarding)	fibs.s (avec forwarding)	daxpy.s (sans forwarding)	daxpy.s (avec forwarding)
Nombre de cycles	153	101	309	224
Nombre d'instructions	78	78	101	101
CPI	1.961	1.294	3.059	2.217
Nombre de cycles d'attente	62	10	193	108

Tableau 5 : Tableau des caractéristiques du pipeline pour les programme fibs.s et daxpy.s

Notre objectif est donc d'optimiser ces résultats (en mode forwarding) afin d'obtenir le nombre de cycles d'attente minimum qui serait idéalement de 0. Pour l'algorithme de Fibonacci, voici ce que nous avons fait :

```
; Compute first twelve Fibonacci numbers and put in array, then print
        .data
 fibs: .space 48          ; "array" of 12 32_bits words to contain fib values
size: .word  12           ; size of "array"
        .text
        addi $t0, r0, fibs    ; load address of array
        lw   $t5, size(r0)    ; load array size
        addi $t2, r0,1        ; 1 is first and second Fib. number
        sw   $t2, 0($t0)      ; F[0] = 1
        sw   $t2, 4($t0)      ; F[1] = F[0] = 1
        addi $t1, $t5, -2     ; Counter for loop, will execute (size-2) times

loop: lw   $t3, 0($t0)        ; Get value from array F[n] -> $t3
        lw   $t4, 4($t0)      ; Get value from array F[n+1] -> *t4
        add  $t2, $t3, $t4    ; $t2 = F[n] + F[n+1]
        sw   $t2, 8($t0)      ; Store F[n+2] = F[n] + F[n+1] in array
        addi $t0, $t0, 4      ; increment address of Fib. number source
        addi $t1, $t1, -1     ; decrement loop counter
        bnez $t1, loop        ; repeat if not finished yet.
        nop
        halt
```

```
; Compute first twelve Fibonacci numbers and put in array, then print
        .data
 fibs: .space 48          ; "array" of 12 32_bits words to contain fib values
size: .word  12           ; size of "array"
        .text
        addi $t2, r0,1        ; 1 is first and second Fib. number
        addi $t0, r0, fibs    ; load address of array
        lw   $t5, size(r0)    ; load array size
        sw   $t2, 0($t0)      ; F[0] = 1
        addi $t1, $t5, -2     ; Counter for loop, will execute (size-2) times
        sw   $t2, 4($t0)      ; F[1] = F[0] = 1

loop: lw   $t3, 0($t0)        ; Get value from array F[n] -> $t3
        lw   $t4, 4($t0)      ; Get value from array F[n+1] -> *t4
        addi $t1, $t1, -1     ; decrement loop counter
        add  $t2, $t3, $t4    ; $t2 = F[n] + F[n+1]
        sw   $t2, 8($t0)      ; Store F[n+2] = F[n] + F[n+1] in array
        addi $t0, $t0, 4      ; increment address of Fib. number source
        bnez $t1, loop        ; repeat if not finished yet.
        nop
        halt
```

Figure 1 : Code Assembleur pour le programme fibs.s : à gauche sans modification et à droite avec réordonnancement

Les principaux changements que nous avons réalisés consistent à réordonnancer afin de limiter les cycles d'attente en y plaçant des instructions comme les addi ou en réduisant les dépendances. Ainsi, nous avons changé la ligne « addi $t1, $t1, -1 » de place.

Fonctions / Caractéristiques	fibs.s (sans forwarding)	fibs.s (avec forwarding)	fibs.s optimisé (sans forwarding)	fibs.s optimisé (avec forwarding)
Nombre de cycles	153	101	122	91
Nombre d'instructions	78	78	78	78
CPI	1.961	1.294	1.564	1.166
Nombre de cycles d'attente	62	10	31	0

Tableau 6 : Tableau des caractéristiques du pipeline pour le programme fibs.s avec et sans optimisation

Pour l'algorithme daxpy, voici ce que nous avons réalisé afin de réduire au maximum les cycles d'attente :

```
;; MIPS program double ax plus y (daxpy)
        .data
A:      .double 10.0
X:      .double 8.0, 1.0,2.0,3.0,4.0,5.0,7.0,8.0,9.0,10.0,11.0,12.0
Y:      .double 3.0,14.0,13.0,4.0,8.0,9.0,8.0,7.0,6.0,5.0,4.0,3.0
N:      .word 12

        .text
main:
        l.d   f0,A(r0)      ; f0 = A
        dadd  r1,r0,r0      ; r1=i=0
        ld    r2,N(r0)      ; r2=N
loop:   l.d   f1,X(r1)      ; f1=x[i]
        l.d   f2,Y(r1)      ; f2=y[i]
        mul.d f1,f1,f0      ; f1=axi
        add.d f2,f2,f1      ; f2=axi+yi
        s.d   f2,Y(r1)      ; y[i]=axi+yi
        daddi r1,r1,8       ; i++
        daddi r2,r2,-1      ; N++
        bne   r2,r0,loop        ;
        nop
        halt
```

```
;; MIPS program double ax plus y (daxpy)
        .data
A:      .double 10.0
X:      .double 8.0, 1.0,2.0,3.0,4.0,5.0,7.0,8.0,9.0,10.0,11.0,12.0
Y:      .double 3.0,14.0,13.0,4.0,8.0,9.0,8.0,7.0,6.0,5.0,4.0,3.0
N:      .word 12

        .text
main:
        l.d   f0,A(r0)      ; f0 = A
        dadd  r1,r0,r0      ; r1=i=0
        ld    r2,N(r0)      ; r2=N

loop:   l.d   f1,X(r1)      ; f1=x[i]
        l.d   f2,Y(r1)      ; f2=y[i]
        mul.d f1,f1,f0      ; f1=axi
        daddi r2,r2,-1      ; N++
        daddi r1,r1,8       ; i++
        add.d f2,f2,f1      ; f2=axi+yi
        s.d   f2,Y(r1)      ; y[i]=axi+yi
        bne   r2,r0,loop    ;
        nop
        halt
```

Figure 2 : Code Assembleur pour le programme daxpy.s : à gauche sans modification et à droite avec réordonnancement

Nous avons principalement réordonnancé le code en changeant l'ordre des instructions (daddi et l.d).

Fonctions / Carac-téristiques	daxpy.s (sans forwarding)	daxpy.s (avec forwarding)	daxpy.s optimisé (sans forwarding)	daxpy.s optimisé (avec forwarding)
Nombre de cycles	309	224	261	200
Nombre d'instructions	101	101	101	101
CPI	3.059	2.217	2.584	1.980
Nombre de cycles d'attente	193	108	145	84

Tableau 7 : Tableau des caractéristiques du pipeline pour le programme daxpy.s avec et sans optimisation

Nous pourrions réduire davantage les cycles d'attente en inversant le « **mul** » et le « **add** » et en optimisant mais le programme ne correspondrait plus à nos attentes. Il ne réaliserait plus le $y_i = a*x_i + y_i$ mais ferait le $y_i = x_i + y_i$, la multiplication serait alors faite mais n'aurait aucune incidence sur le résultat, elle serait simplement stockée f1 et ne serait pas utilisée. Voici le programme hypothétique avec le résultat faussé en sortie :

```
1  ;; MIPS program double ax plus y (daxpy)
2      .data
3   A: .double 10.0
4   X: .double 8.0, 1.0,2.0,3.0,4.0,5.0,7.0,8.0,9.0,10.0,11.0,12.0
5   Y: .double 3.0,14.0,13.0,4.0,8.0,9.0,8.0,7.0,6.0,5.0,4.0,3.0
6  N:  .word 12
7
8      .text
9  main:
10        l.d   f0,A(r0)      ; f0 = A
11        dadd  r1,r0,r0      ; r1=i=0
12        ld    r2,N(r0)      ; r2=N
13
14 loop:  l.d   f2,Y(r1)      ; f2=y[i]
15        l.d   f1,X(r1)      ; f1=x[i]
16        daddi r2,r2,-1      ; N++
17        add.d f2,f2,f1      ; f2=axi+yi
18        mul.d f1,f1,f0      ; f1=axi
19        s.d   f2,Y(r1)      ; y[i]=axi+yi
20        daddi r1,r1,8       ; i++
21        bne   r2,r0,loop    ;
22        nop
23        halt
```

Figure 3 : Code Assembleur optimisé pour le programme daxpy.s afin de réduire au maximum les cycles d'attente (le résultat n'est pas celui attendu)

Fonctions / Caractéristiques	daxpy.s (sans forwarding), résultat faussé	daxpy.s (avec forwarding), résultat faussé
Nombre de cycles	189	141
Nombre d'instructions	101	101
CPI	1.871	1.396
Nombre de cycles d'attente	73 (61 raw stalls + 12 memory not available)	25 (24 raw stalls + 1 memory not available)

Tableau 8 : Tableau des caractéristiques du pipeline pour le programme daxpy.s faussé avec et sans optimisation

3. Ecriture de programmes assembleur

3.1 Premier programme

Maintenant, notre objectif est de réaliser ce premier code C en Assembleur puis de faire un déroulage de boucle :

```c
#define N 12
double A[N], B[N] ;
for(int i=0 ; i<N; i++)
{
if(i&0x1)  // i impair
    B[i]=B[i]+A[i] ;
else       // i pair
    B[i]=B[i]-A[i] ;
}
```

Figure 4 : Code C qui additionne des nombres ou soustrait des nombres en fonction de la parité et les stocke dans B[i]

Voilà comment nous avons réalisé ce programme C en code Assembleur :

```
1       .data
2   A:  .double 8.0, 1.0,2.0,3.0,4.0,5.0,7.0,8.0,9.0,10.0,11.0,12.0
3   B:  .double 3.0,14.0,13.0,4.0,8.0,9.0,8.0,7.0,6.0,5.0,4.0,3.0
4   N:  .word 12
5
6       .text
7   main:
8       ld      r2,N(r0)        ; r2 = N
9       dadd    r1,r0,r0        ; r1 = i = 0
10      dadd    r3,r0,r0        ; r3 = 0
11
12
13  loop:
14      l.d     f0,A(r1)        ; A[i]
15      l.d     f1,B(r1)        ; B[i]
16      andi    r3, r1, 8       ; comparaison impair
17      bnez    r3, impair      ; si impair jump a la fonction impair
18
19      sub.d   f1,f1,f0        ; si pair, B[i] = B[i] - A[i]
20      jal     finPrgm         ; Sauter la fonction impair
21
22      impair:
23      add.d   f1,f1,f0        ; B[i] = B[i] + A[i]
24
25      finPrgm:
26      daddi   r2,r2,-1        ; N++
27      daddi   r1,r1,8         ; i++
28      s.d     f1,B(r0)        ; stocke dans B[i] la valeur adéquate
29      bne     r2,r0,loop
30      halt
```

Figure 5 : Code Assembleur qui additionne des nombres ou soustrait des nombres en fonction de la parité et les stocke dans B[i]

Les seuls points intéressants de ce programme sont le « bnez » qui permet de se rendre à la fonction « impair » si i est impair et l'instruction « jal » qui permet de sauter la fonction « impair » si i est pair.

Puis nous pouvons vérifier que notre code a bien fonctionné en vérifiant la valeur de « b » à la fin du programme :

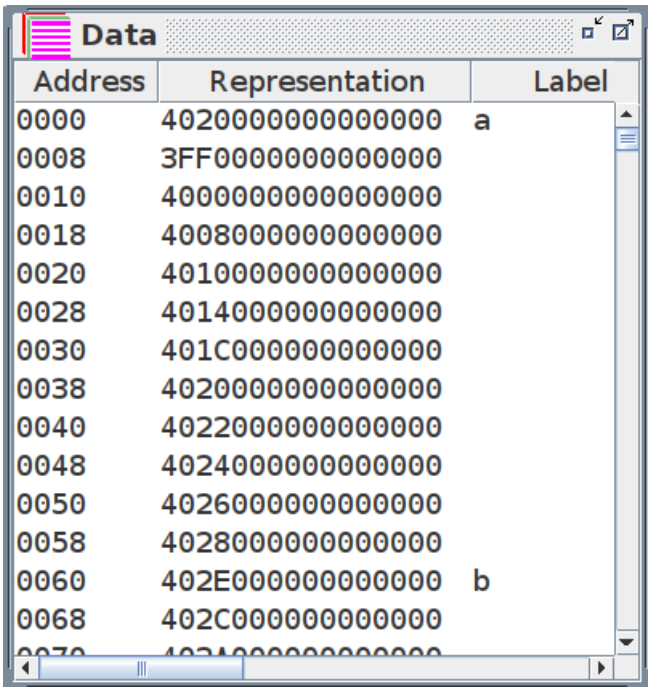


Figure 6 : Vérification du bon du fonctionnement du code Assembleur

Nous voyons que nous avons « b » qui vaut 402E 0000 0000 0000. Pour stocker dans « b », nous avons utilisé l'instruction « s.d ». « s.d » permet de stocker une virgule flottante de 64 bits. Ainsi nous pouvons utiliser un convertisseur en ligne pour convertir cette virgule flottante en décimal :

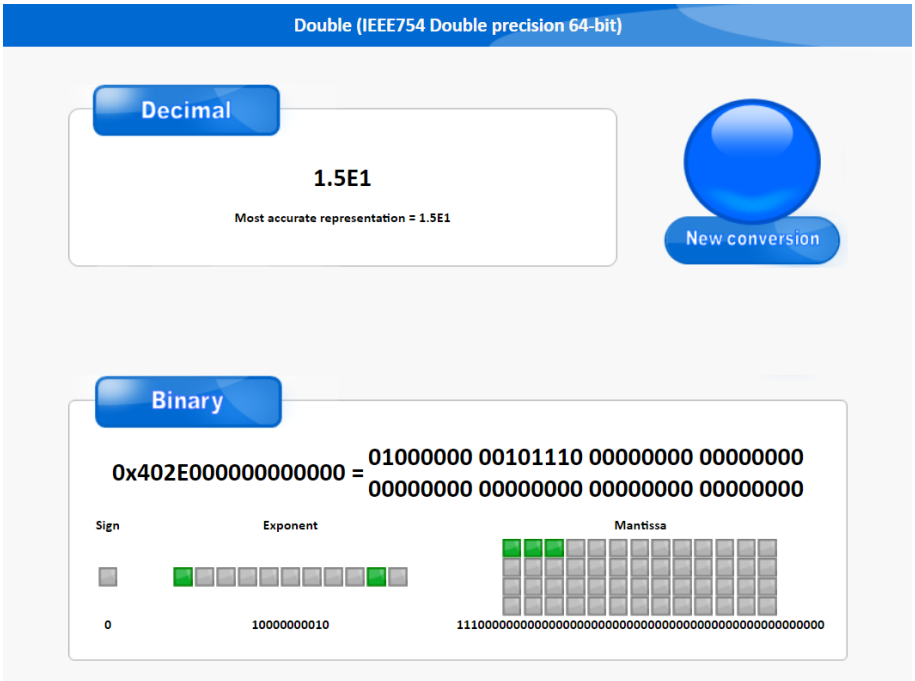


Figure 7 : Convertisseur double 64 bits vers décimal

Nous voyons que « b = 402E 0000 0000 0000 » vaut en réalité 15 en décimal ce qui est correct car lors de la dernière itération, i vaut 11 et est donc impair. Nous devons alors faire B[i] = A[i] + B[i] = 3.0 + 12.0 = 15.0 (derniers éléments du tableau).

Puis nous pouvons regarder le nombre de cycles ainsi que le nombre de cycles d'attente de notre programme. Il s'effectue en 194 cycles sans forwarding et en 157 avec. Il y a 49 cycles d'attentes sans forwarding et seulement 6 avec.

Fonctions / Carac-téristiques	myFunc.s (sans forwarding)	myFunc.s (avec forwarding)
Nombre de cycles	194	157
Nombre d'instructions	118	118
CPI	1.644	1.330
Nombre de cycles d'attente	49 (43 raw stalls + 6 memory not available)	6 memory not available

Tableau 9 : Tableau des caractéristiques du pipeline pour le programme myFunc.s avec et sans forwarding

Nous cherchons maintenant à réduire ce nombre de cycles ainsi que le nombre de cycles d'attente en réalisant un déroulage de boucle d'ordre 2. Voici le nouveau programme assembleur que nous obtenons :

```
1        .data
2   A:   .double 8.0, 1.0,2.0,3.0,4.0,5.0,7.0,8.0,9.0,10.0,11.0,12.0
3   B:   .double 3.0,14.0,13.0,4.0,8.0,9.0,8.0,7.0,6.0,5.0,4.0,3.0
4   N:   .word 12
5
6        .text
7   main:
8        dadd  r1,r0,r0      ; r1 = i = 0
9        ld    r2,N(r0)      ; r2 = N
10       daddi r3,r0,8       ; r3 = i + 1
11
12  loop:
13       l.d     f0,A(r1)        ; A[i]
14       l.d     f1,B(r1)        ; B[i]
15       l.d     f2,A(r3)        ; A[i+1]
16       l.d     f3,B(r3)        ; B[i+1]
17
18       sub.d   f1,f1,f0        ; B[i] = B[i] - A[i], la premiere iteration est pair
19       add.d   f3,f3,f2        ; B[i+1] = B[i+1] + A[i+1]
20
21       daddi   r2,r2,-2        ; N++
22       daddi   r1,r1,16        ; i++
23       s.d     f1,B(r0)        ; stocke dans B[i] la valeur adéquate
24       s.d     f3,B(r0)        ; stocke dans B[i+1] la valeur adéquate
25       bne     r2,r0,loop
26       halt
```

Figure 8 : Code Assembleur qui additionne des nombres ou soustrait des nombres en fonction de la parité et les stocke dans B[i] avec déroulage de boucle d'ordre 2

Avec ce nouveau code, nous obtenons là encore « b = 402E 0000 0000 0000 » soit b = 15 ce qui montre que notre programme est correct. Concernant le nombre de cycles, il est seulement de 97 sans forwarding et 90 avec soit une réduction d'environ 60 à 100 comme le montre le tableau suivant :

Fonctions / Carac-téristiques	myFunc.s (sans forwarding)	myFunc.s (avec forwarding)	myFunc.s (sans forwarding), avec déroulage de boucle d'ordre 2	myFunc.s (avec forwarding), avec déroulage de boucle d'ordre 2
Nombre de cycles	194	157	97	91
Nombre d'instructions	118	118	70	70
CPI	1.644	1.330	1.385	1.3
Nombre de cycles d'attente	49 (43 raw stalls + 6 memory not available)	6 (6 memory not available)	18 (6 raw stalls + 12 memory not available)	12 (12 memory not available)

Tableau 10 : Tableau des caractéristiques du pipeline pour le programme myFunc.s avec et sans déroulage de boucle d'ordre 2 et avec et sans forwarding

Nous observons également des réductions concernant le nombre d'instructions ainsi que le nombre de cycles d'attente.

3.2 Deuxième programme

Maintenant, notre objectif est de réaliser ce deuxième code C en assembleur puis de faire un déroulage de boucle :

```c
#define N 12
double A[N], somme=0.0 ;
for(int i=0 ; i<N; i++){
somme += A[i] ;
}
```

Figure 9 : Code C qui additionne des nombres et les stocke dans une variable « sum »

Voilà comment nous avons réalisé ce programme C en code Assembleur :

```
1          .data
2
3    A:    .double 8.0,1.0,2.0,3.0,4.0,5.0,7.0,8.0,9.0,10.0,11.0,12.0
4    sum:  .double 0.0
5    N:    .word 12
6
7          .text
8    main:
9            l.d   f1,sum(r0)    ; f1 = 0.0 premiere iteration
10           dadd  r1,r0,r0      ; r1 = i = 0
11           ld    r2,N(r0)      ; r2 = N
12
13   loop:
14           l.d   f0,A(r1)      ; f0 = A[i]
15           add.d f1,f1,f0      ; f1 = f1 + A[i], f1 = 0.0 premiere iteration
16           daddi r2,r2,-1      ; N++
17           daddi r1,r1,8       ; i++
18           s.d   f1,sum(r0)    ; stock f1 dans sum
19           bne   r2,r0,loop    ; boucle
20           halt
```

Figure 10 : Code Assembleur qui additionne des nombres et les stocke dans une variable « sum »

Puis nous pouvons vérifier que notre code a bien fonctionné en vérifiant la valeur de « sum » à la fin du programme. Déjà nous pouvons voir que « n = 0000 0000 0000 000C » soit 12 comme ce que nous avons écris dans notre code Assembleur.

Data

Address	Representation	Label
0000	4020000000000000	a
0008	3FF0000000000000	
0010	4000000000000000	
0018	4008000000000000	
0020	4010000000000000	
0028	4014000000000000	
0030	401C000000000000	
0038	4020000000000000	
0040	4022000000000000	
0048	4024000000000000	
0050	4026000000000000	
0058	4028000000000000	
0060	4054000000000000	sum
0068	000000000000000C	n

Figure 11 : Vérification du bon du fonctionnement du code Assembleur

Puis nous voyons que nous avons « sum » qui vaut 4054 0000 0000 0000. Pour stocker dans « sum », nous avons utilisé l'instruction « s.d ». « s.d » permet de stocker une virgule flottante de 64 bits. Ainsi nous pouvons utiliser un convertisseur en ligne pour convertir cette virgule flottante en décimal :

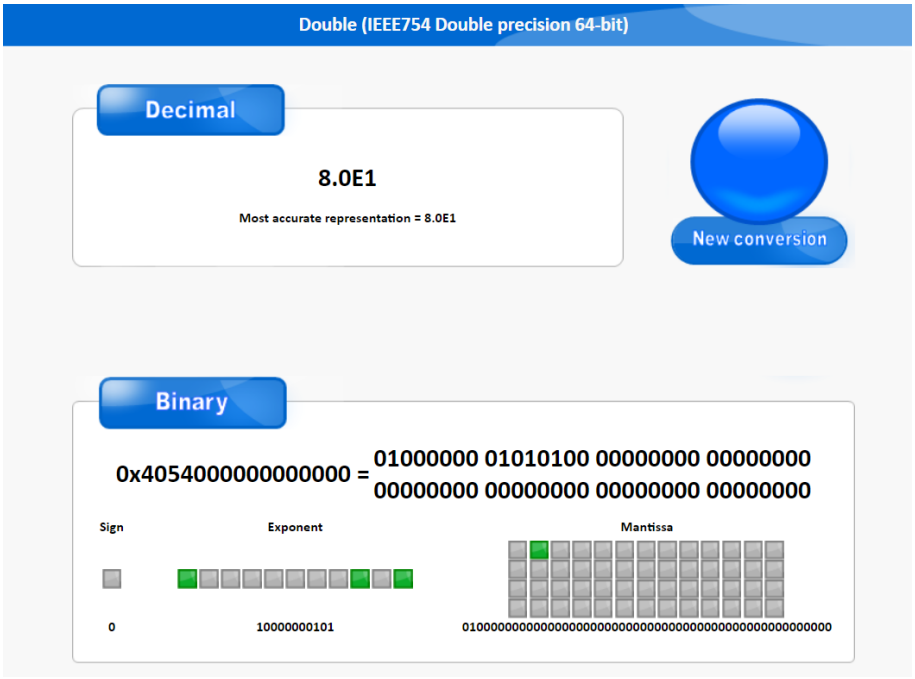


Figure 12 : Convertisseur double 64 bits vers décimal

Nous voyons que « sum = 4054 0000 0000 0000 » vaut en réalité 80 en décimal ce qui est correct si nous faisons la somme des termes du tableau A, nous obtenons : 8+1+2+3+4+5+7+8+9+10+11+12 = 80

Puis nous pouvons regarder le nombre de cycles ainsi que le nombre de cycles d'attente de notre programme. Il s'effectue en 156 cycles sans forwarding et en 115 avec. Il y a 61 cycles d'attentes sans forwarding et seulement 24 avec.

Fonctions / Carac-téristiques	myFunc2.s (sans forwarding)	myFunc2.s (avec forwarding)
Nombre de cycles	156	115
Nombre d'instructions	75	76
CPI	2.0	1.513
Nombre de cycles d'attente	61 (raw stalls)	24 (raw stalls)

Tableau 11 : Tableau des caractéristiques du pipeline pour le programme myFunc2.s avec et sans forwarding

Nous cherchons maintenant à réduire ce nombre de cycles ainsi que le nombre de cycles d'attente en réalisant un déroulage de boucle d'ordre 2. Voici le nouveau programme assembleur que nous obtenons :

```
1         .data
2
3   A:    .double 8.0,1.0,2.0,3.0,4.0,5.0,7.0,8.0,9.0,10.0,11.0,12.0
4   sum:  .double 0.0
5   N:    .word 12
6
7         .text
8   main:
9           l.d   f1,sum(r0)    ; f1 = 0.0 premiere iteration
10          dadd  r1,r0,r0      ; r1 = i = 0
11          ld    r2,N(r0)      ; r2 = N
12
13  loop:
14          l.d   f0,A(r1)      ; f0 = A[i]
15          daddi r1,r1,8       ; i+1
16          add.d f1,f2,f0      ; f1 = sum + A[i], f1 = 0.0 premiere iteration
17          l.d   f2,A(r1)      ; f2 = A[i+1]
18          add.d f2,f2,f1      ; f2 = f1 + A[i+1], f1 = 0.0 premiere iteration
19          daddi r2,r2,-2      ; N+1
20          daddi r1,r1,8       ; i+1
21          s.d   f2,sum(r0)    ; stock f2 dans sum
22          bne   r2,r0,loop    ; boucle
23          halt
```

Figure 13 : Code Assembleur qui additionne des nombres et les stocke dans une variable « sum » avec déroulage de boucle d'ordre 2

Avec ce nouveau code, nous obtenons là encore « sum = 4054 0000 0000 0000 » soit sum = 80 ce qui montre que notre programme est correct. Concernant le nombre de cycles, il est seulement de 116 sans forwarding et 85 avec soit une réduction d'environ 30 à 40 comme le montre le tableau suivant :

Fonctions / Carac-téristiques	myFunc2.s (sans forwarding)	myFunc2.s (avec forwarding)	myFunc2.s (sans forwarding), avec déroulage de boucle d'ordre 2	myFunc2.s (avec forwarding), avec déroulage de boucle d'ordre 2
Nombre de cycles	156	115	116	85
Nombre d'instructions	75	76	58	58
CPI	2.0	1.513	2.0	1.465
Nombre de cycles d'attente	61 (raw stalls)	24 (raw stalls)	49 (raw stalls)	18 (raw stalls)

Tableau 12 : Tableau des caractéristiques du pipeline pour le programme myFunc2.s avec et sans déroulage de boucle d'ordre 2 et avec et sans forwarding

Nous observons également des réductions concernant le nombre d'instructions ainsi que le nombre de cycles d'attente.